/spec reverse

Reverse-engineer a repo: /spec reverse <owner/repo | URL> [–language –type [–stack]] [–create] [–depth] [–folder] [–github | –gitlab]

Overview

`/spec reverse` fetches a public repo's metadata, root file tree, and README via the GitHub or GitLab API, then asks the currently selected LLM model to generate a synthetic creation prompt that could have produced that repo.

The generated prompt is useful as a seed for scaffolding a similar project: copy it into the message window, feed it to another command, chain it straight into spec creation with the `--create <name>` option, or scaffold the project itself with `--folder` (optionally creating a private hosting remote with `--github` / `--gitlab`).

Synthesis runs on the currently selected model, so switch to the model you want before running the command.

Syntax

The syntax lines below read `--language`/`--type`/`--stack` as a single optional group: none of them, or all of `--language` + `--type` (+ optional `--stack`). They are shown separately only so the group is not missed.

```
/spec reverse <owner/repo | URL>
/spec reverse <owner/repo | URL> --create <name>
/spec reverse <owner/repo | URL> --depth <N>
/spec reverse <owner/repo | URL> [--language <lang> --type <type> [--stack <name>]]
/spec reverse <owner/repo | URL> [--language <lang> --type <type> [--stack <name>]] \
    [--folder <path>] [--github | --gitlab]
/spec reverse help
```

Options

Form	Description
`/spec reverse <owner/repo>`	owner/repo shorthand, e.g. `ratatui/ratatui`
`/spec reverse <URL>`	full GitHub URL, e.g. `https://github.com/ratatui/ratatui`
`--language <lang>`	target language for the generated prompt - the same values `/new` accepts
`--type <type>`	target app type: `library`, `cmdline`, `tui`, `gui`, or `webapp`
`--stack <name>`	optional framework stack layer (`axum`, `warp`, `raylib`, `gtk4`, `ratatui`)
`--create <name>`	chain into `/spec create`, auto-generating a spec named `<name>` from the reverse-engineered prompt; with `--folder`, it lands at `<folder>/specs/<name>/`
`--depth <N>`	directory levels to fetch from the repo tree (1-10, default 1)
`--folder <path>`	with scaffold flags, also scaffold the project in `<path>` (created if missing; default: the current directory)

`--github` / `--gitlab`	with scaffold flags, create a private remote, set it as `origin`, and push the initial commit (mutually exclusive)

Both identifier forms are accepted by the same command; pick whichever you have at hand.

`--language`, `--type`, and `--stack` are the `/new` scaffold flags, parsed and validated by the same `/new` parser, so they take exactly the same values and have the same purpose: they steer the generated prompt towards a target project shape. `--language` and `--type` must be supplied together; a lone `--stack` is rejected. Run `/new help` for the full accepted-value lists.

`--folder`, `--github`, and `--gitlab` only take effect when the scaffold flags are present: with them, `/spec reverse` scaffolds a real project using the same engine `/new` and `/spec govcreate` use. `--folder` names the target directory (it must be empty apart from `.ragent/`, `log/`, and `target/`); without it the current directory is scaffolded in place. `--github` / `--gitlab` additionally create a private remote and push the initial commit. Supplying `--folder`, `--github`, or `--gitlab` without `--language` and `--type` is a usage error.

The `<repo>` positional is required. An invalid invocation is never silent: it prints the specific cause followed by this command's usage block. In particular, a flag token placed where `<repo>` belongs (for example `/spec reverse --language rust --type tui`) is reported as a usage error naming the offending token, rather than being treated as a different subcommand or silently dropped, and a flag tail that fails `/new` validation (a missing `--type`, an unregistered `--language`, a duplicate flag, or `--github` together with `--gitlab`) is reported with the shared `/new` error text. Always run the command with `<repo>` first, before any flags.

Examples

Plain owner/repo shorthand:

```
/spec reverse ratatui/ratatui
```

Full GitHub URL:

```
/spec reverse https://github.com/ratatui/ratatui
```

Steer the synthetic prompt to a Rust command-line project:

```
/spec reverse tokio-rs/axum --language rust --type cmdline
```

Steer it to a Rust TUI project using the ratatui stack and chain straight into spec creation:

```
/spec reverse tokio-rs/axum --language rust --type tui --stack ratatui --create config-svc
```

Fetch two levels of the repo tree:

```
/spec reverse tokio-rs/axum --depth 2
```

Scaffold a Rust command-line project in `./my-app` from the reverse-engineered prompt:

```
/spec reverse tokio-rs/axum --language rust --type cmdline --folder ./my-app
```

Scaffold it and create a private GitHub repository for it:

```
/spec reverse tokio-rs/axum --language rust --type cmdline --folder ./my-app --github
```

Output

The message window shows the scaffold summary first (when scaffold flags were supplied), then repo fetch progress, then the generated prompt. With `--create <name>`, the spec creation output follows the prompt.

Workflow

1. Fetch - the GitHub or GitLab API supplies the repo's metadata, file tree, and README.
2. Synthesise - the currently selected LLM model turns that material into a synthetic creation prompt that could have produced the repo.
3. Scaffold (optional) - with `--language`/`--type`, `--folder <path>` runs the `/new` engine in the target folder, and `--github` / `--gitlab` create a private remote and push. The step runs before synthesis so a refusal is reported immediately; a refused scaffold does not stop the prompt generation.
4. Chain (optional) - `--create <name>` hands the prompt to `/spec create`, which auto-generates a spec named `<name>`. With `--folder`, the spec root is the scaffolded project (`<folder>/specs/<name>/`).

Notes

- `--language` / `--type` / `--stack` only steer synthesis; they do not change what is fetched.
- `--language` / `--type` / `--stack` / `--folder` / `--github` / `--gitlab` reuse the `/new` and `/spec govcreate` scaffold engine; the scaffolded project gets the same workspace, docs, git init, and initial commit as `/new`.
- A scaffold refusal (non-empty target folder, or a hosting failure) is reported with `[err]` and the run continues to generate the prompt; nothing is scaffolded in that case.
- `--create <name>` names the spec produced by the chained `/spec create` run; with `--folder` it is written under the scaffolded project rather than the invoking directory.
- `--depth` controls how many directory levels of the repo tree are fetched; the default is 1 (root only).
- The fetch step uses the GitHub or GitLab API, so the target repo must be public.
- A usage error (including a flag where `<repo>` belongs, or a flag tail that fails `/new` validation) prints the cause and the usage block and stops; no API call is made.

Related

- `/spec create` - the command `--create` chains into
- `/new` - the command whose `--language` / `--type` / `--stack` / `--folder` / `--github` / `--gitlab` flags this command reuses
- `/spec govcreate` - the command this reverse/scaffold flow is modelled on